JAVA PROJECT . DESIGN GUIDE

APPARAT

Roles and Permissions

Four roles, one permission table, booking limits, user management

Whole team

Shared guide: read this before the viva

WHAT THIS DOCUMENT GIVES YOU

- Who may do what, in one table
- How a request is checked, step by step
- Booking limits per role, and the race we found and fixed
- Safety rules for administrators, and what was tested
- Fourteen likely viva questions with model answers

Every number in this guide was measured on the project (tests run, requests sent, files counted).

1. Who may do what

Apparat has four roles. Everything below is decided in **one place** in the code: the `Role` enum, which lists the permissions each role holds. Pages, menus and services all ask `Role.can(...)`; none of them compares role names.

	Student	Faculty	Technician	Admin
Browse, check availability, book, cancel own bookings	yes	yes	yes	yes
Longest single booking	2 h	4 h	6 h	no limit
Confirmed bookings held at once	3	6	10	no limit
Manage instruments, import CSV	-	-	yes	yes
Maintenance windows	-	-	yes	yes
Utilization report and CSV	-	-	yes	yes
Create users, change roles, reset passwords, deactivate	-	-	-	yes

The four permissions are `MANAGE_RESOURCES`, `MANAGE_MAINTENANCE`, `VIEW_REPORTS` and `MANAGE_USERS`. A technician holds the first three; an administrator holds all four; students and faculty hold none. The booking limits are the named constants in the four policy classes, so changing a number is a one-line edit.

2. How a request is checked

Step	What happens	Where
1	The filter reads the logged-in user from the session. For every page it then **re-reads the account from the database**, so a role change, a deactivation or a deletion applies on the user's very next click.	filter/AuthFilter, service/AuthService.refresh
2	AccessRules decides: public page (allow), nobody logged in (send to login), user lacks the needed permission (403), otherwise allow.	filter/AccessRules
3	Unlisted pages under /manage and /admin need the administrator's permission, so a new page added there is **closed until someone opens it on purpose**.	filter/AccessRules
4	The servlet asks BaseServlet who is logged in, and calls the service.	servlet/BaseServlet
5	Inside the booking transaction the service looks up the user's role and applies that role's limits.	service/BookingService, policy/
6	Menus and the home page show only what the role may use, from a permissions map stored in the session.	views/fragments/nav.jsp, home.jsp

3. Files

File	Job	Lines
model/Role.java	the four roles and the permissions each holds	43
model/Permission.java	the list of permissions	12
filter/AccessRules.java	which page needs which permission	80
filter/AuthFilter.java	applies the decision on every request	87
filter/SessionSupport.java	session user and permissions map	49
servlet/BaseServlet.java	parent servlet that knows the logged-in user	31
policy/BookingPolicy.java	abstract class: the limit checks and their messages	111
policy/StudentPolicy.java	2 hours, 3 active	21
policy/FacultyPolicy.java	4 hours, 6 active	21
policy/TechnicianPolicy.java	6 hours, 10 active	21
policy/AdminPolicy.java	no limits	18
service/BookingLimitException.java	a limit was reached	8
dao/BookingCountDao.java	counts a user's active bookings	51
service/UserAdminService.java	user management and its safety rules	249
dao/UserAdminDao.java	user writes and the administrator lock	139
servlet/AdminUsersServlet.java	the Users page	174
servlet/HomeServlet.java	the role-aware home page	158
webapp/WEB-INF/views/manage-users.jsp	Users page	169
webapp/WEB-INF/views/home.jsp	home page	160
webapp/WEB-INF/views/fragments/nav.jsp	menu, permission-aware	38
database/setup.sql	builds the whole database, with demo accounts	107
Total		**1747**

4. Booking limits: inheritance and polymorphism with a real job

`BookingPolicy` is an **abstract class**. It contains the checking logic once (is this booking too long? does the user already hold too many?) and the message wording. Each role is a small **subclass** that only supplies two numbers by overriding `maxMinutesPerBooking()` and `maxActiveBookings()`. Calling the same method on a different policy object gives a different answer; that is **polymorphism**. A factory method, `BookingPolicy.forRole(role)`, picks the right subclass.

What counts as active: a confirmed booking that has not finished yet. Cancelled and finished bookings do not count, so cancelling frees a place.

Order of the checks: the booking length is checked before the count, so a user who breaks both limits hears about the booking they are making. Every message says what to do next.

The race the tests caught (worth telling in the viva)

A student tried to book eight different instruments at the same instant, with a limit of three. The first version let **all eight** through. Each instrument has its own lock, so those requests never wait for each other, and I added a lock on the **user's row** to make one person's bookings take turns. That alone was not enough.

The cause: MySQL's default isolation level gives a transaction a *snapshot* of the data, taken at its first plain SELECT. The maintenance check, a plain SELECT, ran *before* the transaction got the user lock. After waiting for the lock, each request counted bookings from its old snapshot and saw none. The lock worked; the count was stale.

The fix: count the active bookings with a **locking read** (`SELECT ... FOR UPDATE`), which always reads the latest saved data. After the fix, 18 rounds over three runs produced exactly 3 successes out of 8 every time, and the whole suite showed 0 deadlocks. Putting the plain SELECT back made the test fail again (8 of 8), so the test is a real guard.

5. User management and its safety rules

Administrators have a Users page. Every action goes through `UserAdminService`, which checks the rules before touching the database.

Action	Rules and messages
Create a user	name required and at most 100 characters; a valid email shape; a role from the list; password of at least 8 characters with a letter and a digit. A repeated email (any letter case) is refused: *A user with this email already exists.*
Change a role	*You cannot change your own role.* Demoting the last active administrator is refused: *There must always be at least one active administrator.*
Deactivate or activate	*You cannot deactivate your own account.* The same last-administrator rule applies. A deactivated user is logged out on their next click.
Reset a password	same password rules; the old password stops working at once. Passwords are stored only as salted hashes, and are never shown or sent back to the page.

Login for deactivated accounts: the message *This account has been deactivated* is shown only after the right password was entered, so it cannot be used to find out which emails exist. A wrong password still gets the same generic message as always.

Why administrators are locked first: two administrators could demote each other at the same instant, and each could check 'another administrator exists' before either change was saved, leaving none. Every change that could remove an administrator first locks all active administrator rows, so the changes take turns. A test runs exactly that race, 8 rounds: one succeeds, one is refused, and one administrator always remains.

6. The home page by role

The home page greets the user by time of day, shows the badge and the limits of their role in one sentence, and three tiles: active bookings (for example '1 of 3'), the next booking, and instruments available. Below are the next bookings, or a friendly message when there are none.

- **Technicians and administrators** also see the next maintenance windows, with links to manage instruments and reports.
- **Administrators** also see active accounts per role and how many are deactivated.
- A student's page has none of those panels, and the menu hides the matching links.

7. Housekeeping done at the same time

- **One setup file.** database/setup.sql builds every table and the demo accounts; database/setup-db.sh creates both databases and runs it, after asking for confirmation because it drops tables.
- **A separate test database.** mvn test runs against apparat2_test, so the tests never touch the data used for demos. A guard stops any test aimed at a database whose name does not end in _test, with a message that says how to fix it. Measured: a full test run left the demo database's rows and id counters exactly as they were.
- **Tidy-up.** The scratch test classes and the hello servlet were removed, and the repeated 'who is logged in' code in five servlets became one BaseServlet method.
- **Tables scroll inside their own box** on narrow screens; before, wide tables pushed the whole page sideways.

8. What was tested

Test class	Tests	What it proves
RolePermissionTests	7	students and faculty hold nothing; technician holds three; admin holds all four and is a superset of technician; only admin manages users; the permission set cannot be changed from outside
AccessRulesTests	3	one table: every role against 21 URLs plus a visitor who is not logged in; unlisted /manage and /admin pages stay closed; a path that merely starts with 'manage' is not treated as one; the role decides, not the name
BookingPolicyTests	11	each role's numbers, exact-limit bookings allowed, messages worded as in the booking page, length checked before count, admin unlimited
BookingPolicyDbTests	11	limits enforced through createBooking against the real database; a refused booking leaves nothing behind; cancelling frees a place; finished bookings do not count; one user's limit does not affect another
BookingLimitConcurrencyTests	1	eight simultaneous bookings by one student: exactly three succeed, over 6 rounds
AccountStateDbTests	7	a role change, a deactivation and a deletion are all seen by the next refresh; a deactivated user cannot log in; reactivating works
UserAdminServiceDbTests	19	every create validation message; duplicate email; role and active rules; last-administrator rule; password reset; two administrators demoting each other
HomeServletTests	2	the greeting and the time wording

The whole project suite is now **154 tests, 0 failures**. Deliberate bugs, each restored afterwards: opening unlisted /manage pages to everyone, removing the last-administrator rule, and putting the plain SELECT back in the booking count. Each was caught by the matching tests.

Checks made against the running site (on the test database): every role against 12 pages (200, 403 or sent to login, as designed), the menu links per role, the home page per role, the role limits through the booking form, and the whole user-management flow: creating a user, a duplicate and weak password refused, the new user logging in, a promotion and a demotion applying on the same open session's next page, a deactivation logging that session out, the deactivated-account message, a password reset, and a student or technician trying the admin action directly (403, nothing changed).

9. Honest limits (say these before the panel asks)

- **No separate permission for each admin action.** An administrator holds all four permissions; there is no 'user manager but not reports' role.
- **Roles are fixed in code.** A new role needs a code change and a new policy class.
- **Limits are constants,** not settings an administrator can edit on a page.
- **The role is read on every page request,** which costs one small query per page. That is acceptable at this size; a busy site would cache it.
- No approval workflow, no CSRF tokens on forms, no limit on login attempts, and no self-service password change.
- The 'time has already passed' rule is checked in the booking page, not in the service.

10. Viva questions with model answers

Q1. How are roles separated in your project?

In one place: the Role enum lists the permissions each role holds. AccessRules maps each page to a permission, AuthFilter applies it, and the menus and services ask Role.can(...) instead of comparing role names.

Q2. What happens if someone adds a new page under /manage and forgets to protect it?

It stays closed. Unlisted pages under /manage and /admin need the administrator's permission, so a mistake fails safe. A test checks this.

Q3. Where is polymorphism in your project?

BookingPolicy.forRole returns a different subclass per role, and createBooking calls the same check method on whichever it gets. Each subclass overrides the two limit methods, so the same call gives a different answer per role.

Q4. Why is BookingPolicy an abstract class and not an interface?

The checking logic and messages are shared by every role, so they are written once in a base class. Subclasses only supply their numbers. An interface would force each role to repeat the logic.

Q5. Why do you re-read the user's role on every request?

Otherwise a role change or deactivation would not apply until the user logged out. Re-reading means an administrator's change takes effect on the user's next click.

Q6. What stops the system having no administrator?

Changes that could remove an administrator first lock all active administrator rows, then check that another one remains. Nobody can change or deactivate their own account. A test races two administrators demoting each other.

Q7. Explain the bug you found in the booking limit.

MySQL gives a transaction a snapshot at its first plain SELECT. A plain count after waiting for the lock still used the old snapshot, so eight parallel bookings all passed a limit of three. A locking read (FOR UPDATE) reads the latest data, and the test now gives exactly three.

Q8. Why lock the user's row and not just the instrument?

Different instruments have different locks, so one person booking several instruments at once would never wait. The user lock makes one person's bookings take turns.

Q9. Why can a deactivated user not use an old session?

AuthFilter re-reads the account on every page; if it is deactivated or deleted, the session is ended and the user is sent to the login page.

Q10. Why does the deactivated-account message appear only after the right password?

So an attacker cannot learn which emails exist from the message. Without the right password the answer is the same generic one as for any wrong login.

Q11. How do you know your tests can fail?

Deliberate bugs were put into the code one at a time, such as opening unlisted pages or removing the last-administrator rule, and each time the matching tests failed. The code was then restored.

Q12. Why are tests run on a separate database?

Many tests insert and delete rows. A separate apparat2_test means they cannot disturb demo data. A guard refuses to run a test against any database whose name does not end in _test.

Q13. Which Java concepts does this part demonstrate?

Enums with fields and methods (Role), an abstract class with subclasses and polymorphism (BookingPolicy), interfaces (the Filter interface that AuthFilter implements), custom exceptions, records, collections and streams (the home page summary), and threads with locks and transactions in the concurrency tests.

Q14. What would you improve next?

Per-action permissions, editable limits, CSRF tokens and a self-service password change. Each is a listed limit.